

UI/UX Breakdown Secret Benefits



Of If I wanted to make a site with the same functions can you break down the ui/Ux
<https://www.secretbenefits.co.uk/>

Do you have access to the site itself so I can replicate its features and functions

Yes. I can access the public-facing site and analyse its structure, flows and UX patterns. I can't access their source code, database, or private backend, but I can break down the functionality closely enough that a developer could recreate it.  Secret Benefits

Overall UX Strategy

Secret Benefits is designed around one objective:

Get users into the member feed as quickly as possible.

Everything else is secondary.

1. Homepage Structure

Hero section

Very simple

- Logo
- Login
- Free Signup
- Large headline
- Gender selection
- Primary CTA

UX Purpose

Reduce cognitive load.

No complicated explanation.

Users immediately identify themselves and start.

2. Onboarding Flow

Step 1

Choose:

- Man
- Woman

Step 2

Create account

Minimal fields:

- Username
- Location
- Email
- Password

Step 3

Upload photos

Step 4

Browse immediately

UX Principle

Progressive disclosure.

Don't ask for everything at once.

3. Tinder-like Member Feed

The real product starts here.

Each card contains:

Photo

Large dominant image.

Small details

- Age
- Location
- Online indicator
- Verification badges

Quick actions

 Favorite

 Message

 Unlock album

UX Principle

Low friction browsing.

No need to open profiles.

4. Profile Page Layout

Left side

Gallery

Public photos

Private album teaser

Center

Biography

About

Interests

Relationship goals

Right side

Sticky action panel:

- Message
- Favorite
- Request private photos

UX Principle

Always keep the CTA visible.

5. Search Experience

The search system is a huge part of retention.

Filters:

- Age
- Distance
- Ethnicity
- Body type
- Height
- Smoking
- Drinking
- Relationship type
- Verification

Users never hit dead ends.

Results update instantly.

6. Gamification

This is very clever.

Profile visitors

"45 people viewed you"

Favourites

"23 members saved you"

Recently online

Top admirer

Verification badges

Secret albums

Creates curiosity and dopamine loops.

7. Messaging UX

Conversation screen resembles Messenger:

Large profile image.

Chat bubbles.

Online status.

Photo sharing.

Typing indicators.

Unread badges.

Key principle

Users shouldn't feel they're "using a dating site".

They should feel they're already communicating.

8. Monetisation UX

They avoid subscriptions.

Instead:

Credits.

Examples:

Unlock photos.

Start conversations.

Become top admirer.

This feels less painful psychologically than:

"£50/month".

9. Mobile First Design

Large cards.

Big buttons.

Minimal text.

Fast scrolling.

Instagram-style experience.

Likely breakpoints:

320px
768px
1024px
1440px

If You Were Building This

Frontend

Next.js + Tailwind

Components:

Navbar
Hero
Signup Wizard
Profile Card
Search Filters
Infinite Member Grid
Profile Page
Messenger
Notifications
Credits System
Dashboard
Settings

Backend

Supabase or Firebase

Tables:

Users

Profiles

Photos

PrivateAlbums

Messages

Favorites

Visitors

Notifications

Credits

Transactions

Reports

Verification

Features

MVP

✓ Sign up

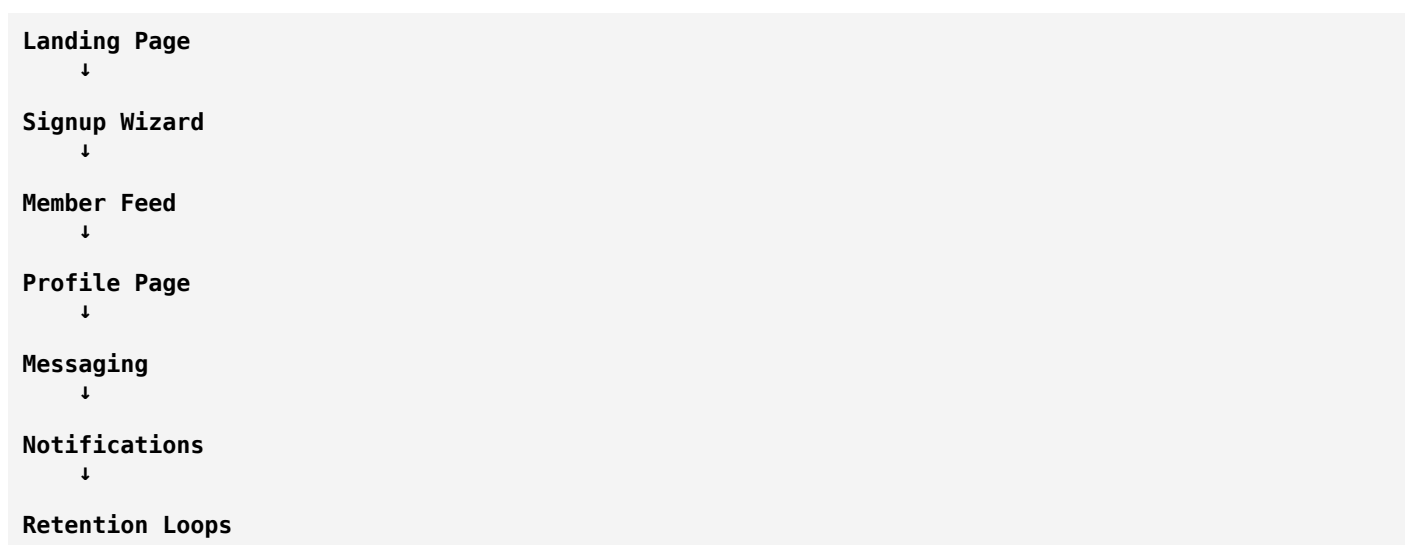
✓ Login

- ✓ Profiles
- ✓ Upload photos
- ✓ Search
- ✓ Filters
- ✓ Messaging
- ✓ Notifications
- ✓ Favorites
- ✓ Online status

Advanced

- ✓ Private albums
- ✓ Verification badges
- ✓ Read receipts
- ✓ Typing indicators
- ✓ Visitors
- ✓ Profile boosts
- ✓ Premium credits
- ✓ Push notifications
- ✓ AI moderation
- ✓ Video verification

UI Hierarchy



If you're building something similar, I could break down every screen individually and create a full 20-30 page UI/UX blueprint, including wireframes and database schema, so a developer could reproduce the entire platform.



I want to build it through Cursor. Can you give me a highly detailed cursor outline that relocated these functions and features so I can build a replica site. Using cursor and vercel

Yes. Cursor + Vercel + Supabase is probably the fastest stack for building something like Secret Benefits. I wouldn't try to clone the site page by page. Instead, build the platform as a collection of systems.

Stack

```
Frontend
-----
Next.js 15
TypeScript
TailwindCSS
Shadcn UI
Framer Motion

Backend
-----
Supabase

Database
-----
PostgreSQL

Storage
-----
Supabase Storage

Realtime
-----
Supabase Realtime

Deployment
-----
Vercel

Payments
-----
Stripe

Images
-----
Cloudflare Images

Moderation
-----
OpenAI API
AWS Rekognition

Email
-----
Resend

Push Notifications
-----
OneSignal
```

Cursor Project Structure

```
app/
```

```
(auth)
  login
  register
  onboarding

(discover)
  discover
  search
  nearby
  favorites

(profile)
  profile/[id]
  edit-profile
  photos
  verification

(messages)
  inbox
  conversation/[id]

(account)
  dashboard
  notifications
  credits
  settings

(admin)
  moderation
  reports
  analytics

components/
lib/
hooks/
types/
supabase/
```

DATABASE

users

```
SQL
id
email
password
created_at
role
credits
is_verified
last_online
online_status
```

profiles

```
SQL
id
user_id
username
age
gender
city
country
headline
```

```
bio
occupation
height
body_type
smoker
drinker
relationship_type
looking_for
```

photos

```
SQL
id
profile_id
url
is_private
sort_order
approved
```

messages

```
SQL
id
sender_id
receiver_id
message
created_at
read_status
```

favorites

```
SQL
id
user_id
favorite_user_id
```

visitors

```
SQL
id
visitor_id
visited_user_id
created_at
```

notifications

```
SQL
id
user_id
type
```

read
content

transactions

SQL
id
user_id
amount
credits
stripe_session

AUTH FLOW

Landing page



Register

email
password
username
gender



Create profile

age
location
headline
bio



Upload photos



Discover page

DISCOVER PAGE

Instagram/Tinder style cards

photo
name age
location
verified badge

online status

buttons

♥ **Favorite**

💬 **Message**

★ **View profile**

Infinite scroll

Supabase pagination

Skeleton loaders

No page numbers

SEARCH SYSTEM

Sidebar filters

Age

Distance

Height

Body type

Relationship goals

Smoking

Drinking

Online now

Verified only

Live query updates.

No refresh.

PROFILE PAGE

Gallery

Public photos

Blurred private gallery

Unlock button

About

Bio
Occupation
Interests
Lifestyle
Looking for

Sticky CTA

Message
Favorite
Request access

CHAT SYSTEM

Realtime websocket
Conversation list
Typing indicators
Read receipts
Photo sending
Emoji support
Online indicators
Push notifications
Unread badges

NOTIFICATION SYSTEM

Someone viewed your profile
Someone favorited you
New message
Photo request
Verification approved

GAMIFICATION

Profile visitors
Popularity score
Top admirer

Daily activity streak

Recently active

Verified badges

Most replied

Photo likes

Trending profiles

CREDIT SYSTEM

Stripe purchases

100 credits

500 credits

1000 credits

Credits spent on

Message requests

Unlock albums

Boost profile

Super favorite

PHOTO SYSTEM

Public images

Private albums

Blur previews

Unlock requests

Watermark protection

Image compression

Cloudflare CDN

ADMIN PANEL

Users

Reports

Messages

Approvals

Photo moderation

Analytics
Revenue dashboard
Ban system
Shadow ban
Verification queue

VERIFICATION SYSTEM

Upload selfie
ID verification
Manual approval
Badge assignment
Verification history

REALTIME FEATURES

Online now
Typing indicators
Read receipts
Seen timestamps
Instant notifications
Message delivery status

COMPONENT LIBRARY

Navbar
MobileBottomNav
SidebarFilters
ProfileCard
PhotoCarousel
ProfileHeader
MessageBubble
ConversationList
NotificationDropdown
CreditCounter
UnlockModal
VerificationBadge

```
OnlineDot
InfiniteGrid
SkeletonCard
EmptyState
ReportModal
```

API ROUTES

```
/api/auth
/api/profile
/api/photos
/api/messages
/api/favorites
/api/visitors
/api/notifications
/api/credits
/api/payments
/api/report
/api/search
/api/verify
```

MIDDLEWARE

Protect routes
Role permissions
Admin access
Rate limiting
Bot detection
Suspicious behaviour detection

VERCEL ENV

```
env
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
OPENAI_API_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
```

RESEND_API_KEY=

Cursor Master Prompt

Paste this into Cursor:

Build a production-ready dating platform inspired by Secret Benefits using Next.js 15 App Router, TypeScript, Tailwind, Shadcn UI, Supabase and Vercel.

Architecture requirements:

Use feature-based folders.

Use server components wherever possible.

Use Supabase Auth.

Use PostgreSQL tables for users, profiles, photos, messages, favorites, visitors, notifications and transactions.

Implement infinite scroll discover feed.

Realtime messaging using Supabase Realtime.

Typing indicators.

Read receipts.

Online presence.

Profile visitors.

Favorites.

Private albums with blurred previews.

Credit system with Stripe.

Notification center.

Search filters with instant updates.

Mobile-first responsive design.

Admin dashboard for moderation.

Image upload with compression.

Skeleton loaders.

Error boundaries.

Loading states.

Rate limiting.

SEO metadata.

Server actions.

Type-safe interfaces.

Zod validation.

React Hook Form.

Reusable components.

Shadcn component system.

```

Dark mode.
Framer Motion animations.
Build incrementally.

Step 1:
Initialize project and install dependencies.

Step 2:
Create database schema.

Step 3:
Build authentication.

Step 4:
Build onboarding flow.

Step 5:
Build discover feed.

Step 6:
Build profile pages.

Step 7:
Build realtime chat.

Step 8:
Build notifications.

Step 9:
Build Stripe credit system.

Step 10:
Build admin dashboard.

Never skip steps.
Always create complete files.
Use best practices and production-level code.

```

I would actually go further and break this into a 50 to 100-page software specification so Cursor can build the entire thing feature-by-feature with much higher accuracy and fewer hallucinations.



Do I just paste these into cursor one by one

No. In fact, that's one of the biggest mistakes people make with Cursor.

You don't want to paste giant prompts repeatedly and hope for the best. Cursor works much better when you treat it like a junior developer and feed it a specification in stages.

1. Create a /docs folder

```

/docs
01-overview.md
02-tech-stack.md
03-database-schema.md
04-auth.md
05-onboarding.md
06-discover-feed.md
07-profile-pages.md
08-search.md
09-messaging.md

```

```
10-notifications.md
11-credit-system.md
12-photo-system.md
13-admin-panel.md
14-verification.md
15-api-routes.md
16-component-library.md
17-ui-guidelines.md
18-deployment.md
```

2. Put the Master Prompt in 01-overview.md

This becomes the project's constitution.

Cursor will reference it automatically.

3. Build one system at a time

Start with:

Step 1

Create project:

```
Bash
npx create-next-app@latest
```

Open Cursor.

Then:

```
Read docs/01-overview.md.
Set up a Next.js 15 app using TypeScript, Tailwind, Shadcn UI and Supabase.
Install all dependencies.
Create feature-based folders.
Do not build any features yet.
```

Let Cursor work.

Commit.

Step 2

Create database:

```
Read docs/03-database-schema.md.
Generate all Supabase SQL tables with foreign keys and indexes.
Create migration files.
Do not create UI yet.
```

Commit.

Step 3

Authentication

```
Read docs/04-auth.md.  
Implement signup, login and middleware route protection using Supabase Auth.  
Create loading states and error handling.  
Do not build profile pages yet.
```

Commit.

Step 4

Onboarding

```
Read docs/05-onboarding.md.  
Build a multi-step onboarding flow.  
  
Username  
Age  
Location  
Bio  
Photo upload  
  
Save progress between steps.  
Mobile first.
```

Commit.

Step 5

Discover Feed

```
Read docs/06-discover-feed.md.  
Build Tinder-style profile cards.  
Infinite scroll.  
Skeleton loaders.  
Favorite button.  
Online status.  
Responsive design.
```

Commit.

Use Git after every major feature

```
Bash
git add .
git commit -m "finished discover feed"
```

This is essential because Cursor will occasionally break things.

Use Composer Mode

Not Agent mode.

Good prompt:

```
Read docs/09-messaging.md.

Implement realtime messaging using Supabase Realtime.

Include:

Conversation list
Typing indicators
Read receipts
Online presence
Image sharing

Use server actions and TypeScript.

Create complete files.

Show me changes before editing.
```

Ask Cursor to inspect code first

Instead of:

Build messaging.

Do:

```
Read docs/09-messaging.md.

Inspect the current project structure.

Explain your plan.

Wait for approval before changing files.
```

This reduces hallucinations dramatically.

Keep every feature in its own markdown file

For example:

09-messaging.md

Markdown

```
# Messaging

## Features

Realtime chat
Typing indicators
Read receipts
Online status
Push notifications
Image uploads
Emoji support
Conversation search

## Tables

messages

id
sender_id
receiver_id
message
created_at
seen

## Components

ConversationList
MessageBubble
TypingIndicator
OnlineBadge
UnreadCounter
```

Cursor performs far better with 2-5 page docs than giant 50-page prompts.

Ideal workflow

```
Docs
↓
Cursor reads docs
↓
Build one feature
↓
Test
↓
Commit to Git
↓
Next feature
↓
Deploy preview to Vercel
↓
Repeat
```

This approach is how many founders are building production SaaS apps with Cursor.

For a project as large as this, I would actually create around 25 detailed markdown specs and a build sequence so Cursor effectively behaves like a full-stack team working through the application in the correct order.



Okay so that.

Yes. That's the approach I'd recommend.

Think of Cursor as a developer that performs best when given **small, well-defined tasks** rather than one giant instruction.

Workflow

1. **Create a /docs folder**
2. Write one markdown file per feature
3. Ask Cursor to read that document
4. Have it explain its plan first
5. Approve the plan
6. Let it generate the code
7. Test the feature
8. Commit to Git
9. Move on to the next feature

Build Order

```
01 Overview
02 Tech Stack
03 Database Schema
04 Authentication
05 Onboarding
06 Discover Feed
07 Profile Pages
08 Search & Filters
09 Messaging
10 Notifications
11 Photo System
12 Credit System
13 Payments
14 Verification
15 Admin Panel
16 API Routes
17 Component Library
18 Deployment
```

The key is that **each document should describe only one system**. Cursor can then keep context and make far fewer mistakes.

For a platform of this size, I'd recommend spending a day writing the specs before writing any code. You'll save many days later.

I can help you create all 18–25 markdown documents in a format specifically optimized for Cursor, with enough detail that you can largely build the platform feature by feature without manually architecting each piece.



Yes. Do that

That's exactly how I'd approach it. You're not building a website, you're building a product. The specs are arguably more important than the code because Cursor will only be as good as the instructions it receives.

I would create **25 spec files** and build the platform in this order:

/docs/01-overview.md

Markdown

Project Overview

Build a production-ready dating platform inspired by Secret Benefits.

Stack:

- Next.js 15
- TypeScript
- Tailwind
- Shadcn UI
- Supabase
- PostgreSQL
- Vercel
- Stripe
- Resend
- Cloudflare Images

Requirements:

- Mobile first
- Server components
- Server actions
- Responsive
- SEO friendly
- Dark mode
- Type-safe
- Loading states
- Error boundaries
- Reusable components

Build one feature at a time.

Never skip steps.

Always inspect existing files before changing code.

/docs/02-project-structure.md

Markdown

Folder Structure

app/

```
(auth)
(discover)
(profile)
(messages)
(account)
(admin)

components/
hooks/
lib/
types/
supabase/
actions/
```

/docs/03-database.md

Contains:

Tables

- users
- profiles
- photos
- private_albums
- messages
- conversations
- favorites
- visitors
- notifications
- reports
- transactions
- subscriptions
- verification_requests

/docs/04-authentication.md

Pages:

```
/login
/register
/forgot-password
/reset-password
```

Features:

- Supabase Auth
- Email verification
- Middleware protection

- Session management
 - Redirect after login
-

/docs/05-onboarding.md

Steps:

Step 1

Gender

Step 2

Username

Age

Location

Step 3

Headline

Bio

Occupation

Step 4

Upload photos

Step 5

Finish profile

Progress saved automatically.

/docs/06-discover-feed.md

Features:

Infinite scrolling.

Profile cards.

Favorite button.

Online status.

Verified badge.

Age and city.

Skeleton loading.

Mobile-first grid.

/docs/07-profile-page.md

Sections:

Gallery

Public photos.

Private album preview.

About

Bio

Occupation

Interests

Lifestyle

Sidebar

Message

Favorite

Unlock album

Report profile

/docs/08-search.md

Filters:

- Age
- Distance
- Height
- Body type
- Drinking
- Smoking
- Relationship goals
- Online now
- Verified only

Instant updates.

No page refresh.

/docs/09-messaging.md

Features:

Realtime chat.

Typing indicators.

Read receipts.

Conversation list.

Unread counter.

Image messages.

Emoji support.

Online presence.

Push notifications.

/docs/10-notifications.md

Notification types:

- New message
- Favorite
- Profile visitor
- Album request
- Verification approved

Dropdown panel.

Unread badge.

Mark all as read.

/docs/11-photo-system.md

Public images.

Private albums.

Blur previews.

Compression.

Drag-and-drop upload.

Delete.

Reorder.

/docs/12-credit-system.md

Credit balance.

Packages:

100 credits

500 credits

1000 credits

Spend credits on:

- Message requests
 - Unlock albums
 - Boost profile
-

/docs/13-payments.md

Stripe integration.

Webhook handling.

Successful purchase flow.

Failed payments.

Transaction history.

/docs/14-visitors.md

Track:

Profile views.

Recently viewed.

View counts.

Daily analytics.

/docs/15-favorites.md

Favorite users.

Remove favorites.

Favorites page.

Count badges.

/docs/16-verification.md

Selfie upload.

ID upload.

Manual approval.

Verified badge.

Verification history.

/docs/17-admin-dashboard.md

Manage:

Users.

Photos.

Reports.

Verification queue.

Revenue.

Analytics.

Shadow bans.

/docs/18-reports.md

Report:

Spam.

Fake account.

Harassment.

Nudity.

Block users.

/docs/19-realtime.md

Supabase realtime:

Presence.

Typing indicators.

Read receipts.

Online users.

/docs/20-api-routes.md

Routes:

```
/api/profile  
/api/photos  
/api/messages  
/api/search  
/api/favorites  
/api/notifications  
/api/payments  
/api/report
```

/docs/21-component-library.md

Components:

```
Navbar  
BottomNav  
ProfileCard  
PhotoCarousel  
FilterSidebar  
ConversationList  
MessageBubble  
NotificationDropdown  
OnlineBadge  
FavoriteButton  
UnlockModal  
CreditCounter  
SkeletonCard  
EmptyState  
ReportModal
```

/docs/22-ui-design-system.md

Shadcn.

8px spacing.

Rounded cards.

Sticky bottom nav.

Large tap targets.

Dark mode.

Framer Motion animations.

/docs/23-security.md

Rate limiting.

Middleware.

Input validation.

Zod.

XSS protection.

Spam prevention.

Bot detection.

/docs/24-deployment.md

Vercel.

Environment variables.

Supabase keys.

Stripe keys.

Resend keys.

Production domains.

/docs/25-build-order.md

```
1 Project setup
2 Database
3 Auth
4 Onboarding
5 Discover feed
6 Profiles
7 Search
8 Messaging
9 Notifications
10 Photo system
11 Favorites
12 Visitors
13 Verification
14 Credits
15 Payments
16 Reports
17 Admin dashboard
18 Deployment
```

Cursor Prompt Pattern

Every time:

```
Read docs/09-messaging.md.
Inspect the existing codebase first.
Explain your implementation plan.
Wait for approval.
Then create complete files using TypeScript, server actions and Shadcn UI.
Do not break existing functionality.
```

This is very close to how a small engineering team would structure a real production build. With these docs and Cursor, you can realistically build something comparable in functionality without needing to hand-code everything yourself.



Great. And when do I set up the other platforms. Needed or does cursor do that?

Cursor is just your AI developer. It doesn't host anything or automatically provision external services. You set those up as they become needed. Don't try to set everything up on day one.

Phase 1: Day One

Set up only:

1. GitHub

Repository for your code.

```
dating-app
```

2. Vercel

For deployment.

Connect GitHub → Vercel.

Every push creates a preview deployment.

3. Supabase

This gives you:

- Database
- Authentication
- Storage
- Realtime websockets

These three services are enough to build the first half of the platform.

Phase 2: After Authentication Works

Add:

Resend

For:

- Email verification
- Password reset
- Welcome emails

Cursor can integrate it, but you'll create the account and API key.

Phase 3: After Messaging Exists

Add:

Cloudflare Images

For:

- Photo uploads
- Compression
- CDN delivery

Alternative:

Use Supabase Storage initially and migrate later.

Phase 4: Monetization

Add:

Stripe

For:

- Credit packages
- Payment processing
- Webhooks

Cursor can build the Stripe integration, but you must create the Stripe account and API keys.

Phase 5: Push Notifications

Add:

OneSignal

For:

- New messages
- Favorites
- Profile visitors

Not essential for MVP.

Phase 6: AI Moderation

Add:

OpenAI API

For:

- Message moderation
- Bio moderation
- Spam detection

Optional:

AWS Rekognition

For image moderation.

Phase 7: Analytics

Add:

PostHog

For:

- Funnel tracking
- User journeys
- Drop-off points

Much better than Google Analytics for SaaS.

Phase 8: Monitoring

Add:

Sentry

For:

- Errors
 - Crashes
 - Performance
-

Timeline

Week 1

- ✓ Cursor
 - ✓ GitHub
 - ✓ Vercel
 - ✓ Supabase
-

Week 2

- ✓ Authentication
 - ✓ Onboarding
 - ✓ Discover feed
-

Week 3

- ✓ Messaging
 - ✓ Profiles
 - ✓ Search
-

Week 4

- ✓ Stripe
 - ✓ Credits
 - ✓ Notifications
-

Month 2

- ✓ Admin dashboard
 - ✓ Verification
 - ✓ AI moderation
 - ✓ Analytics
-

What Cursor Actually Does

Cursor will:

- ✓ Write code

- ✓ Create files
- ✓ Generate SQL
- ✓ Build UI
- ✓ Create API routes
- ✓ Integrate SDKs
- ✓ Refactor
- ✓ Fix bugs

It will not:

- ✗ Create your Stripe account
- ✗ Set up Supabase for you
- ✗ Configure Vercel domains
- ✗ Buy domains
- ✗ Generate API keys
- ✗ Push code to GitHub
- ✗ Deploy automatically without being connected

My recommendation

Start with only:

1. Cursor
2. GitHub
3. Vercel
4. Supabase

Build 80% of the platform first.

Then add:

5. Resend
6. Stripe
7. PostHog
8. Sentry

That sequence is simpler and avoids unnecessary complexity while you're still proving the product.

I would also recommend creating a **30-step build roadmap**, where each step contains the exact Cursor prompt, which external service needs to be connected, and what the expected output should be. That way you can almost work through the project like a checklist.



Can you out the 25 prompts and phases into a single pdf doc

I've created a PDF containing the combined blueprint overview, the 25 development phases, and the platform setup roadmap.

You can download it here:

This version is a concise master blueprint. The next step I'd recommend is creating a much more detailed document (50–100 pages) containing:

- All 25 spec files in full
- Exact Cursor prompts for each phase
- Database schema and SQL
- Folder structure
- UI/UX screen breakdowns
- Component library
- Supabase setup
- Vercel deployment
- Stripe integration
- Realtime messaging architecture
- Admin dashboard
- Security and moderation
- Git workflow
- Testing and launch checklist

Essentially, a complete software specification that you could follow from idea to production while using Cursor as your development team.



How do we attract the first users. And populate the site

For a marketplace or dating platform, the hardest problem isn't the code. It's solving the "empty room problem." People won't stay if there is nobody to interact with.

The solution is to launch in stages, not to the entire UK.

Stage 1: Pick One City

Don't launch everywhere.

Start with:

- London

- Manchester
- Birmingham
- Barcelona (if that's where you're relocating)

People want density.

100 active users in one city is better than 10,000 spread across the country.

Stage 2: Start With One Side

Most marketplaces succeed by focusing on one side first.

Examples:

Uber → drivers first

Airbnb → hosts first

Tinder → women on campuses first

You need to decide who your ideal early users are and build supply first.

Stage 3: Concierge Launch

Don't wait for SEO.

Personally recruit the first 50–100 users.

Use:

- Instagram
- Facebook groups
- Reddit
- Telegram communities
- WhatsApp groups
- Local events

Message people directly.

Stage 4: Create Beautiful Profiles

Before launch, seed the platform with:

- Professional photography
- Complete bios
- Interesting personalities

Never launch with empty profiles.

Stage 5: Invite-Only

Scarcity creates curiosity.

Homepage:

Early Access

Currently launching in London.

Request an invite.

This allows you to collect emails while building density.

Stage 6: Content Engine

Produce content daily:

Instagram reels

TikTok

YouTube shorts

Behind-the-scenes

Success stories

Dating advice

Lifestyle content

The founder's journey

Stage 7: Referral System

Reward users for inviting friends.

Example:

Invite 3 friends

Receive 100 credits

Stage 8: Local Ambassadors

Find 10 super-users.

Give them:

- Free premium
- Profile boosts
- Early access

They become evangelists.

Stage 9: Events

Host:

- Drinks evenings
- Brunches
- Gallery events
- Film screenings
- House parties

Communities form offline.

Stage 10: Paid Ads

Don't start with ads.

Ads work best after:

- Product-market fit
- Retention
- Word of mouth

Otherwise you'll pay for users who immediately leave.

Seeding the Site

You need activity.

User-generated content

Profiles

Photos

Questions

Interests

Likes

Automated content

Daily picks

Trending profiles

New members

Popular members

Recently online

AI Can Help

AI-generated:

- Icebreakers
- Bio suggestions
- Conversation prompts
- Match recommendations

This makes the site feel alive.

Numbers

Your first goal isn't 100,000 users.

It's:

100 active users

↓

500 active users

↓

1,000 active users

↓

10,000 users

The Hidden Secret

Many successful dating apps started by manually doing things behind the scenes.

Founders personally:

- Welcomed users
- Helped people create profiles
- Suggested matches
- Curated communities

The first 100 users are usually earned, not acquired.

The bigger question I'd ask before writing any more code is:

Why would someone use your platform instead of Secret Benefits, Seeking, Bumble, Hinge, or Tinder?

Because your answer to that question will determine how you attract users and what kind of community you should deliberately populate first.



Secret benefits are the main competitors.

For women it's free but men have to buy tokens to pretty much do anything message see Who like them send likes interact so they make all their money from the men the men by credits and use those credits to interact.

I wanna do something different. It's going to be free for the women and the men pay us a monthly membership and that monthly membership allows them unlimited full access to all features. This is how I wanna be Secret benefits Game plan that for me.

Show moreShow less

I think that's actually a strong positioning because you're attacking the biggest complaint users have about Secret Benefits:

"I spent £300 on credits and still don't know what I'm getting."

You're not competing on features. You're competing on trust and simplicity.

Positioning

Secret Benefits:

Women

Free

Men

Pay-per-action

- Message
- Unlock albums
- Read replies
- Send gifts

Psychology:

"Every click costs money."

Your platform:

Women

Free forever

Men

£29.99/month

Unlimited:

- Messages
- Likes
- See profile visitors
- See who likes you
- Private albums
- Video chat
- Search filters

Psychology:

"Pay once. Use freely."

No tokens.

No tricks.

No surprises.

Your Enemy

Not Secret Benefits.

Not Seeking.

Not Tinder.

Your enemy is:

Friction

Anxiety

Hidden costs

Transactional interactions

Brand Message

"Membership. Not Microtransactions."

or

"Unlimited access. One monthly fee."

or

"Connections shouldn't cost credits."

Launch Strategy

Women First

Supply matters.

Nothing else matters.

Without women, there is no product.

Spend 90% of your energy acquiring women.

Women Acquisition

Offer:

Completely free forever

Verified badge

Premium profile

Priority placement

Unlimited messages

Photo verification

Safety features

Women-only support

No ads

Men Waitlist

While building female density:

Create a landing page.

Early Access Membership

Unlimited messages

Unlimited likes

Unlimited everything

£29.99/month

Launching soon

Join waitlist

Collect 1000 emails.

Geographic Launch

One city only.

I would choose:

London.

Not the whole UK.

Initial Numbers

Target:

300 women

100 men

The ratio is everything.

1000 men and 50 women equals disaster.

Revenue

Suppose:

100 paying men

× £29.99

= £2999/month

500 paying men

= £15k/month

1000 paying men

= £30k/month

5000 paying men

= £150k/month

Recurring revenue is vastly more valuable than credit sales.

The Product Difference

Secret Benefits optimizes for:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Revenue per click.

You optimize for:

Time on platform and successful outcomes.

This means:

Video introductions

Voice notes

Verification

Better search

Match suggestions

Real relationships

Safety

Events

Community

Phase 1

Build:

Profiles

Messaging

Search

Notifications

Subscription

Verification

Nothing else.

Phase 2

Introduce:

Voice messages

Video intros

Private albums

AI icebreakers

Live events

Phase 3

Build community.

Brunches.

Networking.

Travel.

Experiences.

Now you're not competing with Secret Benefits.

You're becoming a lifestyle brand.

The biggest opportunity

I think you're making one thing too complicated.

Why should a man pay £30/month before there are enough women?

I'd reverse the launch.

Women

Free forever.

Men

Founding membership.

£9.99/month for life.

Limited to first 500 members.

Then £29.99/month afterwards.

This creates scarcity and gives you recurring revenue while you're small.

If I were building this business with the goal of reaching £100k MRR, I'd focus less on copying Secret Benefits and more on engineering a better economic model and community dynamic. That's where the opportunity really lies.



Great put that game plan into a pdf document please.

I've created the PDF containing the strategic game plan for building and launching your subscription-based dating platform.

 **Download it here:**

The document includes:

- Core positioning against Secret Benefits
- Subscription model and pricing
- Brand messaging
- Women-first acquisition strategy
- One-city launch approach
- Target user ratios
- Founding member pricing strategy
- Phase 1 and Phase 2 features
- Long-term vision
- Revenue milestones and MRR targets

The next logical document would be a much more comprehensive **Founder Playbook (50–100 pages)** covering:

- Brand and positioning
- UI/UX breakdown
- Complete Cursor build specs
- Database architecture
- Marketing strategy
- User acquisition playbook
- Female acquisition funnels
- Male waitlist funnels
- SEO strategy
- Referral loops
- Launch events
- Community-building
- Subscription economics
- Moderation and safety
- Scaling to £100k MRR and beyond

That would essentially serve as the operating manual for the business.



would this be better as an online software or a phone app

I would strongly recommend **web app first, mobile app second**.

Not because apps are bad, but because you're building a marketplace. Your biggest challenge isn't technology. It's getting enough users.

Phase 1: Responsive Web App

Build:

`yoursite.com`

Using:

- Next.js
- Supabase
- Vercel

Make it mobile-first so it feels like an app on phones.

Users can:

- Create profiles
- Upload photos
- Search
- Message
- Subscribe
- Receive email notifications

Advantages:

Faster development

One codebase.

Easier updates

No App Store approvals.

Better SEO

Google can index profiles and content.

Easier payments

Stripe subscriptions avoid Apple and Google fees.

Lower costs

Much less maintenance.

Why I Wouldn't Start With Native Apps

Apps have additional complexity:

Apple review

Dating apps receive extra scrutiny.

Subscription fees

Apple can take 15–30%.

Two codebases

iOS and Android.

Push notifications

More infrastructure.

Store optimization

Another marketing channel to manage.

None of these problems help you get your first 1,000 users.

What I'd Do

Months 1–6

Build a Progressive Web App (PWA).

Make it look and feel like an app.

Users can:

- Add it to their home screen.
- Use it full-screen.
- Receive notifications (Android support is strongest).

Most users won't know it isn't an app.

Months 6–12

If you reach:

- 5,000+ active users
- £20k–£30k MRR

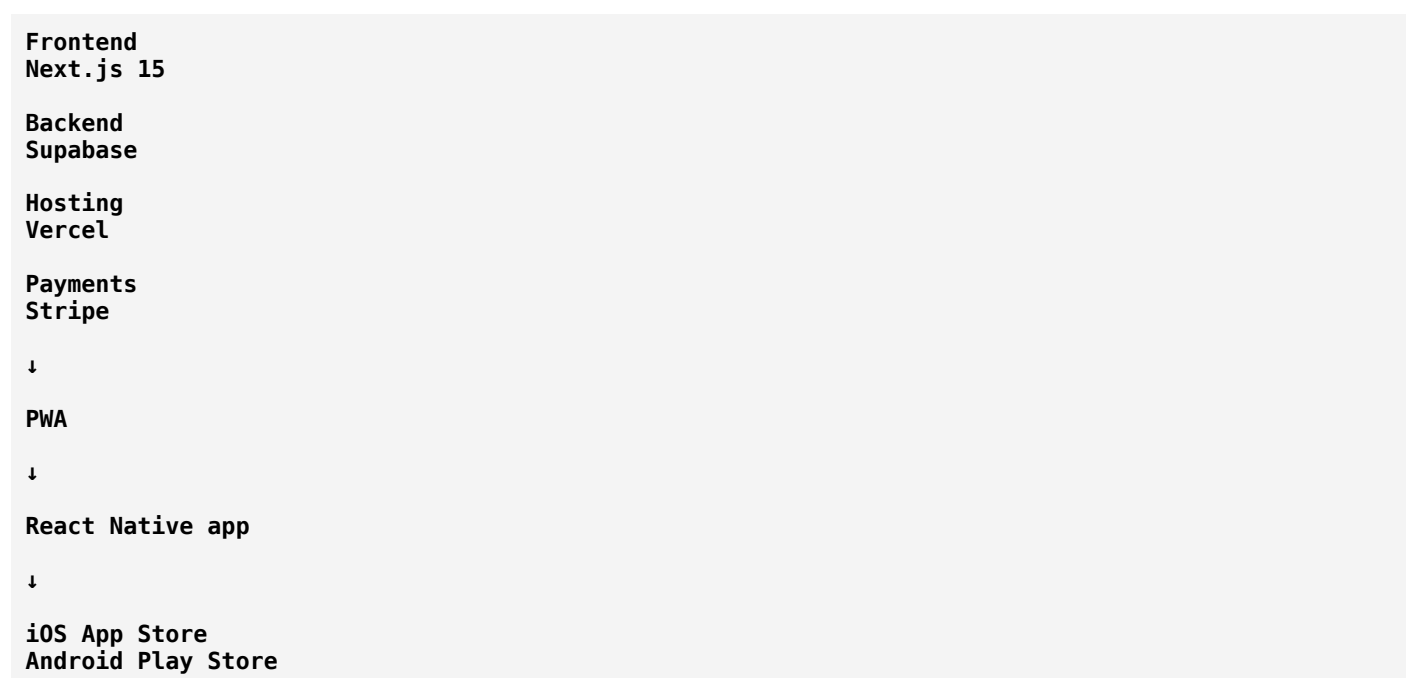
Then build mobile apps.

Use:

React Native + Expo

Because much of your logic can be shared.

Ideal Architecture



What Secret Benefits Does Well

People think they're competing with an app.

They're not.

They're competing with:

- Speed
- Simplicity
- Mobile experience

You can provide all of that through a web app.

My recommendation

Step 1

Build a beautiful mobile-first website.

Step 2

Turn it into a PWA.

Step 3

Reach 1,000 paying members.

Step 4

Build iOS and Android apps.

Step 5

Eventually become:

```
Website
+
PWA
+
iOS App
+
Android App
```

The website should be the foundation. If you do this well, you could postpone building native apps for a year or more and still grow effectively.



so to start i just past the first prompy to cursor or do i need to set up vercel first

No. Don't open Cursor and paste prompts immediately. Spend 30–60 minutes setting up the foundation first. Here's the order I'd recommend.

Step 1: Create a GitHub Repository

Create:

```
dating-platform
```

This will hold all your code.

Step 2: Create a Vercel Account

Connect Vercel to GitHub.

You don't need to deploy anything yet, but getting this connected early makes life easier.

Step 3: Create a Supabase Project

Name:

```
dating-platform-prod
```

Copy and save:

- Project URL
- Anon Key
- Service Role Key

You'll need these later.

Step 4: Install Cursor

Create an empty folder:

```
dating-platform/
```

Open it in Cursor.

Step 5: Paste Prompt #1

Start small:

```
Build a Next.js 15 application using TypeScript, Tailwind CSS and Shadcn UI.
```

```
Use a feature-based folder structure.
```

```
Create:
```

```
app/
```

```
(auth)  
(discover)  
(profile)  
(messages)  
(account)  
(admin)
```

```
components/  
hooks/  
lib/  
types/
```

```
Install all dependencies.
```

```
Do not build features yet.
```

```
Inspect the project first.
```

Explain your plan before making changes.

Let Cursor do the setup.

Step 6: Test Locally

Run:

```
Bash
npm run dev
```

Make sure it works.

Step 7: Push to GitHub

```
Bash
git init
git add .
git commit -m "Initial setup"
git branch -M main
git remote add origin YOUR_REPO
git push -u origin main
```

Step 8: Deploy to Vercel

Import the GitHub repo.

Vercel will automatically deploy.

You should now have:

```
dating-platform.vercel.app
```

Step 9: Back to Cursor

Now move onto Prompt #2:

Database.

Then:

Prompt #3:

Authentication.

Then:

Prompt #4:

Onboarding.

And so on.

Your Actual Timeline

Today

- ✓ GitHub
- ✓ Vercel
- ✓ Supabase
- ✓ Cursor
- ✓ Prompt #1

Tomorrow

- ✓ Database
- ✓ Auth

Day 3

- ✓ Onboarding

Day 4

- ✓ Discover feed

Day 5

- ✓ Profile pages

Think of Vercel and Supabase as the land and utilities.

Cursor is the builder.

You need to buy the land first, then bring in the builder.



I already have a vercel account and cursor is already installed

Perfect. You're much closer than you think.

Here's exactly what I would do next.

1. Create a New GitHub Repo

Name:

```
dating-platform
```

(or whatever your brand name is).

2. Create a Local Folder

```
dating-platform
```

Open that folder in Cursor.

3. Create a /docs Folder

Inside your project:

```
dating-platform/  
docs/
```

Create:

```
01-overview.md  
02-project-structure.md  
03-database.md  
04-authentication.md  
05-onboarding.md
```

Don't worry about all 25 yet.

4. Create a Supabase Project

Go to:

<https://supabase.com>

Create:

```
dating-platform-prod
```

Copy:

- Project URL
- Anon key
- Service Role key

Save these somewhere.

5. Now Start Cursor

Paste this as Prompt #1:

```
Build a Next.js 15 application using TypeScript, Tailwind CSS and Shadcn UI.  
Use a feature-based architecture.  
Create:  
app/  
  
(auth)  
(discover)  
(profile)  
(messages)  
(account)  
(admin)  
  
components/  
hooks/  
lib/  
types/  
  
Install all required dependencies.  
Configure ESLint and TypeScript.  
Do not build features yet.  
Inspect the project first.  
Explain your plan before making changes.
```

Approve the changes.

6. Run

```
Bash  
npm run dev
```

Verify everything loads.

7. Push To GitHub

```
Bash  
git init  
git add .  
git commit -m "Initial setup"  
git branch -M main  
git remote add origin YOUR_REPO  
git push -u origin main
```

8. Connect Vercel

Since you already have Vercel:

- Import GitHub repo
- Deploy

You'll now have:

```
yourdomain.vercel.app
```

9. Return To Cursor

Prompt #2:

```
Read docs/03-database.md.

Create Supabase SQL schema for:

users
profiles
photos
messages
favorites
visitors
notifications

Include indexes and foreign keys.

Do not build UI.

Explain your plan first.
```

The key point:

You do NOT need Vercel fully configured before starting.

Your order is:

```
Cursor
↓
Local project
↓
GitHub
↓
Vercel deployment
↓
Supabase database
↓
Authentication
↓
Onboarding
↓
Discover feed
↓
Profiles
↓
Messaging
```

If I were sitting beside you, I would spend the next hour getting only these four things done:

- ✅ Create GitHub repo
- ✅ Create Supabase project
- ✅ Open empty folder in Cursor
- ✅ Run Prompt #1

Nothing else.

Then tomorrow we'd build the database and authentication. That's where the real app begins.



i want to call the brand www.suggaplay.com

I actually like **SuggaPlay** as a brand.

It feels:

- Modern
- Memorable
- Playful
- Less transactional than "Sugar Daddy" sites

It suggests fun rather than money, which gives you room to build a broader lifestyle and dating brand.

I also checked whether **suggaplay.com** is obviously in use. I couldn't find an active website using that exact domain, although you should verify its registration with a domain registrar before committing.



I would position it like this

SuggaPlay

"Where successful people meet without the games."

Or

"Unlimited connections. One membership."

The name should stand for simplicity:

- Women join free
- Men pay one monthly membership
- Unlimited conversations
- No credits
- No tokens
- No hidden costs

That becomes your core differentiator.

Brand personality

I'd avoid making it feel overtly like a "sugar dating" platform. Instead, aim for:

- Premium
- Stylish
- Discreet
- Confident
- Modern

Think more **Soho House** than "adult dating site."

That opens the door to features like:

- Verified members
- Luxury experiences
- Travel
- Networking
- Relationships
- Mentorship
- Dating

without boxing yourself into one niche.

One thing I'd do before designing anything

Search for:

- trademark conflicts in the countries you plan to operate
- availability of:
 - suggaplay.com
 - @suggaplay on Instagram
 - @suggaplay on TikTok
 - @suggaplay on X
 - @suggaplay on Facebook

Having a consistent brand across your domain and social accounts will make launch much smoother.

I have one suggestion

If you're serious about turning this into a real company, don't just build a website.

Build **SuggaPlay** as a complete brand.

That means creating:

1. Brand strategy

2. Visual identity (logo, colours, typography)
3. Design system
4. Complete UI kit
5. Cursor software specification
6. Marketing strategy
7. Launch strategy
8. Investor deck
9. Product roadmap
10. Growth playbook

That gives you everything from branding through to engineering and customer acquisition, rather than just a functioning website.